

Technical Onboarding Documentation

- ID-only vs Fully managed / Role-based services
 - ID-only (Authentication only)
 - Fully managed / Role-based (Authentication and Authorization managed by us)
- How services can be hidden
 - Hidden from a user's "Services" page (ID-only)
 - Hidden from approvers/users from being able to request access (Optional for role-based, always enabled for ID-only)
 - Hidden from our support console (Optional for role-based, always enabled for ID-only)
 - Hidden from our help page (Optional, enabled by default for ID-only)
- Service configuration information
 - Home URL
 - Post password-reset URL (Optional)
 - Client ID
 - Redirect URL(s)
 - Logout redirect URL(s) (Optional)
 - Response types
 - Token endpoint authentication method
- OIDC scopes
- Application login flows
 - ID-only
 - Fully managed / Role-based
 - Technical overview of the different OIDC flows
 - Authorization Code flow (Response type set to code):
 - Implicit flow (Response type set to id_token or id_token & token)
 - Hybrid Flow (Response type set to code & id_token, code & token, and code & id_token & token)
- Logging out a user
- Public API authentication
- Useful URLs

ID-only vs Fully managed / Role-based services

ID-only (Authentication only)

ID-only services do not require different levels of access to be assigned through DfE Sign-in, all they require is that user has an account, and usually an organisation associated with it. This is useful for applications such as forms or information retrieval services where there are no different access levels, the only thing the service needs to know is the user has a DfE Sign-in account and is linked to X organisation.

Another valid use case for ID-only services is if your service has a CRM that manages user and organisation data. For instance, when a user logs in to your service you can get the user's email and their chosen organisation's UKPRN, and do further processing in your CRM.

Fully managed / Role-based (Authentication and Authorization managed by us)

Role-based services get the same information as ID-only services but have an extra layer of authorization permissions, which we call roles. Roles are different access areas or levels for a service, such as access to different sections of your app, or admins and users. Access to roles is defined through policies, which can limit access to specific organisation categories and types, or even specific users or UKPRNs (speak to our team before/during onboarding to discuss what requirements we can cater to).

When a user linked to an organisation logs in to your application, then you will be able to query our [Public API](#) to get the roles they have on their account, by checking the role codes returned, as they will be static across environments whereas names *could* be more fluid.

This is useful for applications that have dashboard pages where different items can only be accessible if a user has a specific role associated with their account, or to limit access to one type or organisation, or even to allow headteachers to have a different access level

then normal organisation staff for instance.

How services can be hidden

Hidden from a user's "Services" page (ID-only)

For ID-only services, when a user logs in to your service for the first time, they are linked to a hidden organisation on our side, so when they next log in, your service will be listed on the user's services page and they can click on it to go to your homepage.

We can disable this from our side if you would like, so users can only navigate to your service from your homepage or direct links.

Hidden from approvers/users from being able to request access (Optional for role-based, always enabled for ID-only)

For role-based services, we can hide the service from approvers and end-users of organisations, which means approvers can't grant access to it, and end-users can't even request it. It will not be visible for them until they have access, and the only way access can be granted is through our internal Support Console, or through the Manage Console by any users with the "Service Support" role.

This may be useful if you only desire a small number of users, or one nominated user in each organisation to have access to your service, or if you want the service onboarded but do not want it to be visible to users yet. The organisation's approvers and other users wouldn't even be able to request it through our UI, or even know it exists unless it is added to them through the manage console.

Hidden from our support console (Optional for role-based, always enabled for ID-only)

Our support console is only used by our internal team to:

- Map users to organisations and services.
- Check access when we receive support tickets.
- Deactivate users/invitations if required, such as a long period of inactivity.

If you would like your service to not be visible in those checks, or for it to not exist in the list of mappable services, we can hide that for you as well. (If we do hide your role-based service from support, it will need to be hidden from our help page as well, as our support team won't be able to check any access.)

Hidden from our help page (Optional, enabled by default for ID-only)

Our contact form at <https://help.signin.education.gov.uk/contact-us> has a list of services users can request support for, so we can verify their access or check their organisation against your service's policies (role-based only).

We can hide your service from our list if you would like to handle those kinds of requests on your side, or for ID-only services, we hide your service by default to cut out the middleman if there are any errors in your application.

Service configuration information

Whilst these values won't be configurable by your team in our Manage Console until the service is onboarded, if we can get some of this information ahead of time then we can pre-fill these values for your team to give you a head-start.

Home URL

The homepage/landing page URL of the service, and where the link on a user's services page will take them.

For example: `https://www.example.service.gov.uk/home`

Post password-reset URL (Optional)

The URL of the page users will be redirected to after resetting their password. This is optional and if one isn't provided the user will be redirected to `https://services.signin.education.gov.uk/` (or the environment's equivalent).

For example: `https://www.example.service.gov.uk/reset-home` (could also be the same as the Home URL)

Client ID

The OIDC client ID, which is passed with every authentication request to OIDC and is unique to your service. This can be assigned by us, but if you would like to suggest one then they must:

- Be a minimum of 8 characters long.
- Contain only alphanumeric characters (A-Z, a-z, 0-9) and hyphens (-).

These should not be easy to guess, as even though we have a randomly generated client secret for security purposes, it's still nice to have your service's OIDC "username" to not be immediately obvious to users outside of DfE Sign-in. So it could be part of your service's name for example "Service to Check Students Test Results and Certifications" could become "checkStudentResults" or similar. Of course, when a user logs in to your service, they will be able to see this in the address bar or their DevTools network tab, that's why we have the randomly generated client secret that only your team should know!

Redirect URL(s)

A redirect URL is where you want to redirect users to after they have been authenticated. DfE Sign-in will send OIDC authentication response to this URL. You can add as many redirect URLs as you would like, and responses will only be sent to URLs in this list.

URLs must:

- Be HTTPS.
- NOT INCLUDE:
 - Query strings e.g. `?foo=bar`.
 - Localhost (for Production only, lower environments can).
 - Wildcards e.g. `https://*.example.service.gov.uk`.

Logout redirect URL(s) (Optional)

A logout redirect URL is where you want to redirect users once they log out, this could be back to the service homepage or a special "logged out" page. You can add as many logout redirect URLs as you would like, and users will only be able to redirect to URLs in this list.

URLs must:

- Be HTTPS.
- NOT INCLUDE:
 - Query strings e.g. `?foo=bar`.
 - Localhost (for Production only, lower environments can).
 - Wildcards e.g. `https://*.example.service.gov.uk`.

Response types

Based on the OIDC client library used by your application, or how you've configured it, you may require specific OIDC response types. On DfE Sign-in you are allowed to have any combination of the following:

- `code`
- `token`
- `id_token`

And depending on your combination, the OIDC authentication flow will automatically be chosen. See [section 3 of the OIDC specification](#) for more information.

Token endpoint authentication method

This will depend on the OIDC client library used by your application, or how you've configured it. This decides what authentication method your application will use when calling our token endpoint. We have the following options:

- `client_secret_basic`: Your OIDC client will send credentials using the HTTP Basic authentication scheme.
- `client_secret_post`: Your OIDC client will send credentials in the body of a HTTP POST request.

OIDC scopes

When your application calls our OIDC provider, you can request scopes to control the information returned in the DfE Sign-in OIDC response.

We have the following scopes available:

- `openid` (required): Adds the `sub` and `sid` fields to the response, which are the DfE Sign-in user and session IDs.
- `profile`: Adds the `given_name` and `family_name` fields to the response, which are the user's first and last names.
- `email`: Adds the `email` field to the response, which is the user's email.
- `organisation` (required for role-based services): Adds the `organisation` field to the response, which is a JSON object of the user's chosen organisation or `{}` if the user has no organisation linked to their account.

Here is an example of a response with all the scopes included (**organisation type for establishments has the same format as the organisation category, with the type id and the name**):

```
1 {
2   sub: 'B2A79669-3C3F-4E29-98AF-70A80D4F214D',
3   organisation: {
4     id: '3DE9D503-6609-4239-BA55-14F8EBD69F56',
5     name: 'Department for Education',
6     category: { id: '002', name: 'Local Authority' },
7     urn: null,
8     uid: '123',
9     upin: null,
10    ukprn: null,
11    establishmentNumber: '001',
12    status: { id: 1, name: 'Open' },
13    pimsStatus: null,
14    closedOn: null,
15    address: null,
16    telephone: null,
17    statutoryLowAge: null,
18    statutoryHighAge: null,
19    LegacyId: '1031237',
20    companyRegistrationNumber: null,
21    DistrictAdministrativeCode: null,
22    DistrictAdministrativeCode: null,
23    providerTypeName: null,
24    LegalName: null
25  },
26   sid: 'UmhG_1VEShbwzNCQ07imd',
27   given_name: 'Jared',
28   family_name: 'Marsh',
29   email: 'jared.marsh@education.gov.uk'
30 }
```

Scopes Key

- openid (required)
- organisation (optional)
- profile (optional)
- email (optional)

Application login flows

ID-only

1. A user goes to your application's landing page, either through a direct link or through their "Services" page if it isn't hidden, and click on your "sign in" button.
2. Your application will then contact our OIDC provider, and the user will be redirected to DfE Sign-in to:
 - a. Log in if they haven't already.
 - b. Select an organisation on their account, if they have more than 1 organisation and the `organisation` scope is requested by your application.

3. Our OIDC provider will return a JSON response, in the same format defined in the “OIDC scopes” section.
4. Your application can then process or check the user/organisation information however you wish, for example to render pages, or to check against your own internal allow list.

Fully managed / Role-based

1. A user goes to your application's landing page, either through a direct link or through their “Services” page if it isn't hidden, and click on your “sign in” button.
2. Your application will then contact our OIDC provider, and the user will be redirected to DfE Sign-in to:
 - a. Log in if they haven't already.
 - b. Select an organisation on their account, if they have more than 1 organisation and the `organisation` scope is requested by your application.
3. Our OIDC provider will return a JSON response, in the same format defined in the “OIDC scopes” section.
4. Your application **must** then use the information in that provided response to call our Public API to [get the user's access to the service, on any page](#) that requires a user to have a role in order to access it. We advise you to do this on any page requiring that authorization in case a role is moved or a user's access is modified mid-session.

Technical overview of the different OIDC flows

This section will most likely be covered by an OIDC client package/library you will be using, **so these intricacies are not required knowledge**. But for a deeper technical understanding, please feel free to take a look at the flow your application uses.

Authorization Code flow (Response type set to `code`):

The Authorization Code Flow returns an Authorization Code to the Client, which can then exchange it for an ID Token and an Access Token directly. This provides the benefit of not exposing any tokens to the User Agent and possibly other malicious applications with access to the User Agent. The Authorization Server can also authenticate the Client before exchanging the Authorization Code for an Access Token. The Authorization Code flow is suitable for Clients that can securely maintain a Client Secret between themselves and the Authorization Server.

– OIDC Spec Section 3.1 (https://openid.net/specs/openid-connect-core-1_0.html#CodeFlowAuth)

1. To make an authorisation request, your service sends the user to the `/auth` endpoint.
2. The user logs in (or creates an account if they do not have one) and proves their identity. DfE Sign-in lets your user know how their data will be shared with your service, you can view our Privacy Policy at: <https://interactions.signin.education.gov.uk/privacy>.
3. DfE Sign-in returns an authorisation code to your service.
4. Your service makes a token request to the `/token` endpoint, including your client secret (generated after service setup) and includes the authorisation code in the request.
5. Your service receives one or more of: an ID token (containing the user attributes/claims), an access token (to be used in step 6 & 7), and a refresh token (if requested) in the response.
6. If your service requires the claims again i.e. they are not stored in the user session on your service's side or you want additional claims, your service makes a request to the `/me` endpoint with the access token to retrieve user information.
7. Your service receives a response containing user attributes/claims, which you can then use.

Implicit flow (Response type set to `id_token` or `id_token & token`)

 The Implicit Flow is mainly used by **Clients implemented in a browser** using a scripting language. The Access Token and ID Token are returned directly to the Client, which may expose them to the End-User and applications that have access to the End-User's User Agent. **The Authorization Server does not perform Client Authentication.** 

– OIDC Spec Section 3.2 (https://openid.net/specs/openid-connect-core-1_0.html#ImplicitFlowAuth)

⚠️ Only use the implicit flow for applications that specifically require it e.g. your service cannot store secrets, as they can be subject to malicious attacks such as browser vulnerabilities because the ID and access tokens are stored on the user's browser, this **should not be used for new applications without an explicit need for it!** ⚠️

1. To make an authorisation request, your service sends the user to the `/auth` endpoint.
2. The user logs in (or creates an account if they do not have one) and proves their identity. DfE Sign-in lets your user know how their data will be shared with your service, you can view our Privacy Policy at: <https://interactions.signin.education.gov.uk/privacy>.
3. Your service receives ID token and, *if requested*, an access token.
4. Your service validates the ID token and retrieves the user attributes/claims from it, which you can then use.
5. If you requested an access token and require the user's claims again i.e. they are not stored in the user session on your service's side or you want additional claims, then this can sent in a request to the `/me` endpoint to retrieve the user information.
6. Your service receives a response containing the user attributes/claims, which you can then use.

Hybrid Flow (Response type set to `code & id_token`, `code & token`, and `code & id_token & token`)

When using the Hybrid Flow, some tokens are returned from the Authorization Endpoint and others are returned from the Token Endpoint.

– OIDC Spec Section 3.3 (https://openid.net/specs/openid-connect-core-1_0.html#HybridFlowAuth)

The hybrid flow allows an access token and an ID token to be returned from the authorization endpoint, like in the implicit flow, but then allows your service to request another access token and a refresh token from the token endpoint, like in the authorization code flow.

1. To make an authorisation request, your service sends the user to the `/auth` endpoint.
2. The user logs in (or creates an account if they do not have one) and proves their identity. DfE Sign-in lets your user know how their data will be shared with your service, you can view our Privacy Policy at: <https://interactions.signin.education.gov.uk/privacy>.
3. Your service receives a code to be used with the authorization endpoint, an access token to allow access to the `/me` endpoint, and an ID token to receive the user attributes/claims immediately so you can use them.
4. This part of the flow depends on which combination of response types are requested:
 - a. `code & id_token`:
 - i. You can use the user attributes/claims within the `id_token` immediately.
 - ii. Your service can make a token request to the `/token` endpoint, including your client secret (generated after service setup) and includes the authorisation `code` in the request.
 - iii. Your service receives one or more of: an ID token (containing the user attributes/claims), an access token (to be used in step 5 & 6), and a refresh token (if requested) in the response.
 - b. `code & token`:
 - i. Your service can make a token request to the `/token` endpoint, including your client secret (generated after service setup) and includes the authorisation `code` in the request.
 - ii. Your service then receives one or more of: an ID token (containing the user attributes/claims), an access token (to be used in step 5 & 6), and a refresh token (if requested) in the response.
 - iii. Your service can also make a request to the `/me` endpoint using the `token` when required, to retrieve the user's information.
 - iv. Your service then receives a response containing the user attributes/claims, which you can then use.
 - c. `code & id_token & token`:
 - i. You can use the user attributes/claims within the `id_token` immediately.
 - ii. Your service can make a token request to the `/token` endpoint, including your client secret (generated after service setup) and includes the authorisation `code` in the request.
 - iii. Your service receives one or more of: an ID token (containing the user attributes/claims), an access token (to be used in step 5 & 6), and a refresh token (if requested) in the response.
 - iv. Your service can also make a request to the `/me` endpoint using the `token` when required, to retrieve the user information.

- v. Your service then receives a response containing the user attributes/claims, which you can then use.
5. If your service requires the claims again i.e. they are not stored in the user session on your service's side or you want additional claims, your service makes a request to the `/me` endpoint with the access token to retrieve user information.
6. Your service receives a response containing user attributes/claims, which you can then use.

Logging out a user

Users can be logged out of their OIDC session by your application's OIDC client library calling our `end_session_endpoint`. This is defined in our OpenID configuration file located at:

- Dev: <https://dev-oidc.signin.education.gov.uk/.well-known/openid-configuration>
- Test: <https://test-oidc.signin.education.gov.uk/.well-known/openid-configuration>
- Pre-Prod: <https://pp-oidc.signin.education.gov.uk/.well-known/openid-configuration>
- Prod: <https://oidc.signin.education.gov.uk/.well-known/openid-configuration>

Your OIDC client library, if it has a logout or end session function, will probably add any additional GET parameters required when calling that URL. However, if it doesn't, these are the GET parameters that URL will expect:

- `post_logout_redirect_uri` - One of the post logout redirect URIs for your application (configured in the service configuration, **required**)
- And one or both of:
 - `id_token_hint` - If you are using the `id_token` OIDC response type, or have access to the ID token returned by OIDC when using the `code` response type, then the token can be sent to log the user out of your application.
 - `client_id` - The client ID of your application, used to verify the `post_logout_redirect_uri` matches the ones configured for your application.

Public API authentication

One common issue we have relating to authenticating with our public API, is the format the JWT is supposed to have when requesting access to the service, and where the appropriate secrets are.

This is outlined in our [Public API documentation](#) but a visual example is often easier to understand, so here is one from [JWT.io](#):

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJZT1VSX0NMSUV0VF9JRCIsImF1ZCI6ImNpZ25pb15lZHVjYXRpb24uZ292LnVrIn0.Q0mvycjhMuV60Dg950vgXr3quY-WmoCfWhY1s7YFQ-8
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{  "alg": "HS256",  "typ": "JWT"}
```

PAYLOAD: DATA

```
{  "iss": "YOUR_CLIENT_ID",  "aud": "signin.education.gov.uk"}
```

VERIFY SIGNATURE

```
HMACSHA256(  base64UrlEncode(header) + "." +  base64UrlEncode(payload),  YOUR_API_SECRET)
```

☐ secret base64 encoded

As you can see:

- `aud` is `signin.education.gov.uk` - This stays constant across the environments.
- `iss` a.k.a issuer is your client ID - Which you can retrieve from the "Service configuration" page of our Manage console, or ask our team to provide this for you.

- And in the signature, we have your API secret - Which is also available in the “Service configuration” page of our Manage console, but is not to be confused with your client secret!

Your API secret can be regenerated in the Manage console’s “Service configuration” page, by scrolling down to the bottom (position may change), and clicking “Regenerate” next to the secret, and then clicking on “Continue” to ensure that the update is put through to our database.

API

Read the [service configuration help page](#) to find out more about API.

API secret

A value that is created automatically by the system and acts as a password for the DfE Sign-in public API. You can regenerate this value.

[Regenerate](#)

Useful URLs

- OIDC specification: https://openid.net/specs/openid-connect-core-1_0.html
- DfE Sign-in OIDC Provider URLs:
 - Dev: <https://dev-oidc.signin.education.gov.uk/>
 - Test: <https://test-oidc.signin.education.gov.uk/>
 - Pre-Prod: <https://pp-oidc.signin.education.gov.uk/>
 - Prod: <https://oidc.signin.education.gov.uk/>
- DfE Sign-in Public API URLs:
 - Dev: <https://dev-api.signin.education.gov.uk/>
 - Test: <https://test-api.signin.education.gov.uk/>
 - Pre-Prod: <https://pp-api.signin.education.gov.uk/>
 - Prod: <https://api.signin.education.gov.uk/>
- DfE Sign-in Public API documentation: <https://github.com/DFE-Digital/login.dfe.public-api/blob/develop/README.md>
- DfE Sign-in ServiceNow service portal: https://dfe.service-now.com/serviceportal?id=sc_cat_item&sys_id=0c00c1afdb6bc8109402e1aa4b961937&sysparm_category=2f6e34afdb6bc8109402e1aa4b9619aa